IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

Name: SAP Prabhashana (Group 80)	
Student Reference Number: 10967100	

Module Code: PUSL 2021	Module Name: Computing Group Project
Coursework Title: Interim Report Submission	
Deadline Date: 19/02/2026	Member of staff responsible for coursework: Mr. Anton Jayakodi
Programme: BSc (Hons) Data Science , BSc (Hons) Artificial Intelligence	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

Name	Student ID
SAP Prabhashana	10967100
Rubasinghe D D	10967104
YUS De Yapa	10967122
NPMMD Pathirana	10967063
RRI Dissanayaka	10967081
AVJA Amarasekera	10967076
PGT Sewmini	10967111

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group:

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***
Signed :

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.

I *have used/not used translation software.

If used, please state name of software……

Overall mark _____% **Assessors Initials _____** **Date – 19/02/2026**

Acknowledgements

Guidance provided throughout the project was sincerely appreciated. Special gratitude is extended to our project supervisor, Mr. Anton Jayakodi, whose continuous encouragement and strong support played a major role in shaping the final outcome of this work. Although the project requirement was to develop a single application, his motivation and confidence in the team inspired the decision to go beyond that expectation and successfully deliver two applications under the LinkAble project: LinkAble Connect and LinkAble Vision. Appreciation is also extended to the module leader, Mr. Diluka Wijesinghe, for his guidance and support during the course of the module. Their guidance, encouragement, and constructive feedback were instrumental in helping bring the LinkAble project to its final successful completion.

Abstract

LinkAble was developed to address two accessibility problems within one final-year group project: the need for clearer obstacle awareness support for blind and low-vision users, and the need for a practical sign-first communication workflow for users of Sri Lankan Sign Language. The project was ultimately delivered as two focused applications rather than as a single cloud-heavy accessibility platform. LinkAble Vision was implemented as a mobile camera-based obstacle-awareness application that captured frames at controlled intervals, submitted them to a FastAPI and YOLO backend, and returned concise spoken guidance derived from object type, position, and approximate distance. LinkAble Connect was implemented as an Android custom keyboard with a companion onboarding application and a Telegram bot that converted deterministic sign tokens into Sinhala text and converted Sinhala text into a stitched sign image using curated mappings and image assets. An iterative, Agile-inspired process was followed so that early ambitions could be reduced into functions that were demonstrable, testable, and maintainable within the academic timeframe. The resulting project demonstrated that two accessibility problems could be addressed through modest but functional software: a working spoken obstacle-awareness flow in Vision and a linguistically grounded sign-input and bot-mediated communication flow in Connect. Although the final system remained bounded in scope, the delivered artefacts showed a clear, technically coherent, and ethically safer basis for future expansion.

Table of Contents

Acknowledgements *3*

Table of Contents *4*

List of Tables *6*

Chapter 01 - Introduction *7*

1.1 Background **7**

1.2 Problem Statement **7**

1.3 Aim of the Project **8**

1.4 Objectives **8**

1.5 Scope of the System **8**

1.6 Structure of the Report **9**

Chapter 02 - Literature Review *10*

2.1 Introduction **10**

2.2 Mobility Support, Object Detection, and Spoken Guidance **10**

2.3 Sign-First Communication and Structured Accessibility Workflows **11**

2.4 Messaging Mediation and the Value of Controlled Conversion **12**

2.5 Research Gap and Justification for LinkAble **12**

Chapter 03 - System Analysis *14*

3.1 Introduction **14**

3.2 Current Context and Existing Situation **14**

3.3 Stakeholders and User Groups **14**

3.4 Feasibility Analysis **15**

3.5 SWOT Analysis **16**

3.6 Risk Analysis **16**

4.1 Overview **18**

4.2 Requirements Elicitation and Correction Approach **18**

4.3 Functional Requirements **18**

4.3.1 LinkAble Vision Functional Requirements 18

4.3.2 LinkAble Connect Functional Requirements 19

4.3.3 Shared Platform and Data Requirements 20

4.4 Non-Functional Requirements **20**

4.5 Traceability **21**

Chapter 05 - System Architecture *22*

5.1 Introduction **22**

5.2 LinkAble Vision Architecture **22**

5.3 LinkAble Connect Architecture 23
5.4 Technology Stack and Data Assets 24
5.5 Architectural Security, Reliability, and Boundaries 25
Chapter 06 - Method of Approach / Methodology *27*
6.1 Introduction 27
6.2 Development Model Used 27
6.3 How the Methodology Was Applied 27
6.4 Engineering Approach 28
6.5 Tools Used in Development and Collaboration 28
6.6 Team Task Allocation 29
Chapter 07 - System Testing and Quality Assurance *30*
7.1 Introduction 30
7.2 Testing Strategy 30
7.3 Vision Testing 31
7.4 Connect Testing 32
7.5 Quality Assurance Through Report Correction 33
7.6 Limitations of the Final Testing Position 33
Chapter 08 - Conclusions *35*
Team Plan and Responsibility Matrix *36*
SAP Prabhashana (10967100) 36
Rubasinghe D D (10967104) 36
YUS De Yapa (10967122) 37
NPMMD Pathirana (10967063) 37
RRI Dissanayaka (10967081) 38
AVJA Amarasekera (10967076) 38
PGT Sewmini (10967111) 39
Appendices *42*
Appendix A - Source Code Submission 42
Group 80 - LinkAble Vision Source Code 42
Appendix B – Github Source Code 42
Appendix C – Previous Submissions 42
Appendix C – Previous

List of Tables

Table 1 - Project objectives and measures 8
Table 2 - Final delivered system scope 8
Table 3 - Comparison of mobility-support approaches 11
Table 4 - Comparison of sign-first communication approaches 11
Table 5 - Research gap summary 13
Table 6 - Stakeholder analysis 14
Table 7 - TELOS feasibility summary 15
Table 8 - SWOT analysis 16
Table 9 - Risk register 16
Table 10 - Functional requirements - LinkAble Vision 18
Table 11 - Functional requirements - LinkAble Connect 19
Table 12 - Shared platform and data requirements 20
Table 13 - Non-functional requirements 20
Table 14 - Requirements traceability 21
Table 15 - Vision architecture components 22
Table 16 - Connect architecture components 24
Table 17 - Technology stack 25
Table 18 - Data and content assets 25
Table 19 - Development phases 27
Table 20 - Development and collaboration tools 28
Table 21 - Testing levels and purpose 30
Table 22 - Vision functional test summary 31
Table 23 - Connect functional test summary 32
Table 24 - Evidence-led corrections applied to earlier report claims 33

Chapter 01 - Introduction

1.1 Background

Digital accessibility tools often solve only one part of a user problem. Navigation software may give general direction cues yet miss immediate obstacles, while messaging software may support text or speech without preserving sign-first expression. LinkAble was conceived to address these two barriers under one project umbrella: safer environmental awareness during movement and more practical sign-mediated communication.

Within the scope, LinkAble Vision became a camera-based obstacle-awareness application. It opened with a short spoken countdown, activated the rear camera, captured images at controlled intervals, transmitted those images to a local or network-accessible backend, and then played spoken guidance based on what the backend detected. The backend used a YOLO model and a rule-based guidance layer to describe obstacle location as left, center, or right and to estimate distance as near, medium, or far. The value of the Vision application therefore lay in scene awareness and concise spoken feedback.

LinkAble Connect became an Android sign-input workflow and a coversational platform. The uploaded implementation contained a custom Android keyboard service, multiple sign pages, a companion application for setup and help, and a Telegram bot. Together these artefacts supported a sign-oriented communication chain: the keyboard inserted deterministic sign tokens into a text field, the companion application helped the user enable the keyboard and open Telegram, and the bot translated known tokens into Sinhala text or generated a composite sign image from Sinhala input. The Connect workflow could be described as a linguistically complete final product within its defined sign inventory.

1.2 Problem Statement

The first problem addressed by LinkAble concerned environmental awareness during movement. Blind and low-vision users can benefit from navigation systems, yet safe movement also depends on knowing what is immediately present in the path ahead. A curb, person, chair, vehicle, or other obstruction may be more urgent than a broad route instruction. In low-resource contexts, a mobile application that can convert visual detections into understandable speech is therefore valuable even when it does not replace all route planning requirements.

The second problem concerned sign-first communication. Standard messaging applications remain text-first and voice-first, which means that users who prefer to think and communicate through signing are often forced to adapt their meaning into a less natural medium. For Sri Lankan users in particular, there was no comparable structured solution designed around Sri Lankan Sign Language (SLSL) to support sign-first digital communication. Unstructured sticker sharing and ad hoc images were insufficient because they did not preserve stable encoding, did not support repeatable testing, and did not scale well when message content had to be converted, reviewed, or reused. A structured input system with deterministic tokens and controlled conversion was therefore required.

1.3 Aim of the Project

The aim of the project was to design and develop two focused accessibility applications under the LinkAble project umbrella: a spoken obstacle-awareness application for blind and low-vision users, and a sign-input plus Telegram conversion workflow for Sri Lankan Sign Language communication. The aim was pursued through an implementation that remained demonstrable, bounded, and academically defensible within the available time and technical resources.

1.4 Objectives

The final objectives were rewritten so that each could be traced to a demonstrable outcome rather than to an early-stage aspiration.

Table 1 - Project objectives and measures

ID	Objective	Primary measure
OBJ-01	Deliver a final report that matched the uploaded implementation artefacts.	Feature claims aligned to evidence
OBJ-02	Provide LinkAble Vision as a working mobile obstacle-awareness flow with spoken guidance.	Successful session launch, detection response, and speech output
OBJ-03	Provide LinkAble Connect as a practical sign-input and Telegram conversion workflow.	Successful keyboard entry and bot-based conversion
OBJ-04	Maintain deterministic mappings so that known input produced stable output.	Repeatable token-to-text and text-to-sign results
OBJ-05	Keep the final scope feasible, testable, and clearly bounded.	Documented exclusions and traceability across chapters

1.5 Scope of the System

The final delivered system intentionally excluded several features that appeared in earlier planning documents. This was not treated as a project weakness but as a scope correction. The final system was defined around what was uploaded and verifiable: Vision delivered camera-driven obstacle awareness and spoken feedback; Connect delivered Android keyboard entry, a companion setup application, and a Telegram conversion bot.

Table 2 - Final delivered system scope

Area	In scope	Out of scope for final claim

LinkAble Vision	3-second spoken countdown, rear camera session, frame capture every 3.5 seconds, FastAPI and YOLO obstacle detection, left/center/right guidance, near/medium/far estimation, spoken English alerts, inactivity timeout and manual session end.	Turn-by-turn GPS routing, rerouting logic, SOS location dispatch, offline on-device model execution, custom-trained mobility dataset claims.
LinkAble Connect	Android custom IME, alphabetic hand-sign keys, four Sinhala sign pages, deterministic token insertion, transient last-sign preview, guide and FAQ screens, Telegram group bot, JSON mappings, Pillow-based sign image generation, final token and mapping hardening.	Delivered iOS keyboard extension, inline bot operation, automatic speech-to-text, text-to-speech, full-sequence predictive suggestion engine as a final headline feature, unconstrained machine translation.
Shared platform logic	Common project identity, mapping governance, curated PNG sign assets, report-level traceability across proposal, F&TS, interim, and final code.	A full cloud core handling user accounts, histories, or enterprise synchronization across both products.

1.6 Structure of the Report

The report followed the required final-report structure. Chapter 02 reviewed the relevant literature for obstacle awareness and sign-first communication. Chapter 03 analysed the current context, stakeholders, feasibility, risks, and limitations. Chapter 04 translated the scope into functional and non-functional requirements. Chapter 05 described the technical architecture of each application. Chapter 06 explained the development methodology and the way scope changed during implementation. Chapter 07 summarised testing, quality assurance, and the corrections applied to earlier claims. Chapter 08 concluded the report by stating what had been achieved.

Chapter 02 - Literature Review

2.1 Introduction

The literature relevant to LinkAble fell into two broad streams. The first concerned mobility support for blind and low-vision users, especially the relationship between navigation, object detection, and usable audio feedback. The second concerned communication accessibility for Deaf, hard-of-hearing, and non-verbal users, especially the difference between text-first communication tools and sign-first interaction methods. Because the final project contained two separate applications, the literature review was intentionally balanced across those two domains.

Research and industry practice have shown that disability support tools are often fragmented. Visual assistance applications commonly emphasise OCR, scene description, or route narration. Communication tools commonly emphasise text captioning, speech synthesis, or large-scale translation. The design space occupied by LinkAble was narrower. Vision was concerned with converting scene content into brief obstacle guidance, and Connect was concerned with stable sign-token entry plus controlled conversion. The most relevant literature was therefore the literature that discussed constrained, reliability-oriented accessibility design rather than general-purpose AI ambition.

2.2 Mobility Support, Object Detection, and Spoken Guidance

Research on visual impairment and mobility repeatedly highlighted the connection between loss of environmental awareness and reduced independence. Studies on falls, isolation, and employment risk among visually impaired users demonstrated that safe navigation is not only a travel problem but also a wider quality-of-life problem. This made scene-aware support attractive because it could complement existing orientation strategies instead of replacing them.

Commercial and research systems have explored different ways to provide mobility help. Some systems emphasised audio landmarks or route narration, as illustrated by work around Microsoft Soundscape. These tools showed that non-visual navigation cues could reduce cognitive load, but they also depended strongly on map quality, geolocation availability, and route interpretation.

A different line of work focused on camera-based object detection. Mobile computer-vision research demonstrated that lightweight detection models could run with increasing efficiency on constrained devices. This distinction was particularly important for the LinkAble Vision scope. The uploaded application did implement a detection-to-guidance chain. The relevant literature therefore pointed to three design concerns: first, scene interpretation must be translated into actionable speech; second, message frequency must be controlled to avoid noise; and third, the system should remain understandable even when running with modest hardware or network assumptions. Those themes justified why the final Vision application used a guidance layer built on direction and distance categories rather than verbose scene narration.

Table 3 - Comparison of mobility-support approaches

Approach	Strengths	Common limitations	Relevance to LinkAble Vision
Audio landmark and route tools	Strong orientation support and familiar travel metaphors.	Limited awareness of immediate obstacles and dependence on mapping quality.	Helpful conceptual background, but broader than the delivered Vision scope.
Generic object-detection apps	Direct scene awareness through camera input.	Can be noisy, delayed, or difficult to interpret as movement advice.	Closest technical precedent for Vision.
Mobile edge-computer-vision systems	Low-latency inference and reduced dependence on constant cloud calls.	Still require careful message design and device-aware optimisation.	Relevant to the aspiration for lightweight spoken obstacle awareness.

2.3 Sign-First Communication and Structured Accessibility Workflows

Communication accessibility tools are frequently built around writing and speech rather than around the structure of signed languages. Captions and speech-based features can reduce some barriers, but they do not create a sign-first experience and often force meaning into a less natural language mode.

Alternative and augmentative communication research showed that structured non-speech input can be highly valuable when the interface respects the user's preferred mode of expression (ASHA, 2023; Tager-Flusberg and Kasari, 2022). Although AAC research does not directly solve SLSL communication, it helps explain why deterministic symbol systems, constrained vocabularies, and predictable outputs remain useful in practice. The Connect workflow fit this logic: it preferred deterministic token entry and controlled bot conversion over unconstrained automatic translation.

Sign-language technology research often aims at ambitious recognition or generation systems. Systems that use deep neural networks for sign recognition have shown promising results, but they depend on large datasets, careful annotation, and language-specific representation decisions (SignAll Technologies, 2023). Those requirements become much harder when the target language is a lower-resourced sign language and when the delivery environment is a normal chat application. For SLSL, the challenge is not only recognition quality but also the lack of a mainstream digital encoding pathway.

The Connect implementation therefore occupied a deliberately narrower and safer design space. Instead of claiming general machine translation, it used a keyboard-mediated sign code system and a bot-mediated conversion system. This design choice aligned with literature that favours constrained workflows where correctness and repeatability matter more than broad but unreliable automation. The project's use of mappings, preprocessing, postprocessing, and sign-image rendering showed how a modest but dependable communication chain can still create meaningful accessibility value.

Table 4 - Comparison of sign-first communication approaches

Approach	Strengths	Common limitations	Relevance to LinkAble Connect
Text chat with captions	Widely available and easy to deploy.	Not sign-first and linguistically shifts the user into written language.	Represents the baseline that Connect attempted to improve.
Video signing	Natural visual expression and rich human communication.	Bandwidth heavy, asynchronous, and difficult to standardise or search.	Useful in many contexts, but not ideal for lightweight routine messaging.
Automatic sign recognition or generation	Research-rich and potentially expressive.	Data hungry, error prone for low-resource languages, and difficult to control in messaging scenarios.	Important for future work, but beyond the delivered scope.
Deterministic symbol or token workflows	Repeatable, testable, and safer under constrained vocabularies.	Less expressive than open translation and dependent on maintained mappings.	The most direct conceptual basis for the final Connect system.

2.4 Messaging Mediation and the Value of Controlled Conversion

The Connect implementation also related to literature on using existing communication platforms rather than attempting to replace them entirely. Users often adopt accessibility tools more easily when those tools sit beside familiar workflows. A mediated platform approach reduces onboarding friction because the user can continue working in an ordinary chat environment. It also allows accessibility support to be selectively triggered rather than universally imposed.

This mediated strategy has trade-offs. It must respect privacy, avoid silent intervention in private messages, and present clear boundaries between what is local and what is sent to an external service. In the Connect implementation, the keyboard handled token entry and the Telegram bot handled explicit conversion. That separation aligned with the literature's concern for user control. It also offered a safer communication model because unsupported content could be flagged or left unrendered rather than being automatically mistranslated.

2.5 Research Gap and Justification for LinkAble

The literature identified several gaps that justified the final project. First, mobility-support tools often separated abstract navigation from immediate obstacle interpretation. Second, camera-based assistive tools could generate more detections than a user could safely process. Third, sign-language communication tools often prioritised large-scale recognition or generic captions rather than practical sign-first authoring in mainstream messaging. Fourth, low-resource language and regional context issues remained significant, especially for Sri Lankan sign workflows.

The LinkAble deliverables addressed these gaps in bounded ways. Vision focused on concise obstacle guidance instead of claiming full autonomous navigation. Connect focused on a structured keyboard and mediated conversion path instead of claiming universal translation. The contribution of the project

therefore lay not in feature maximalism but in delivering two concrete accessibility workflows with traceable software artefacts and reporting discipline.

Table 5 - Research gap summary

Gap area	Observed weakness in prior work	LinkAble response
Immediate mobility awareness	Navigation guidance and obstacle interpretation are often treated separately.	Vision translated scene detections into short spoken obstacle guidance.
Usable alerting	Detection-heavy systems may overload the user with repeated labels.	Vision used rule-based direction and distance messages plus speech suppression.
Sign-first routine messaging	Most mainstream tools remain text-first or video-heavy.	Connect provided deterministic sign-token input inside ordinary text fields.
Low-resource SLSL tooling	Regional sign workflows lack stable encoding paths and maintained digital assets.	Connect used curated mappings, sign PNG assets, and bot-based conversion.

Chapter 03 - System Analysis

3.1 Introduction

System analysis was carried out to understand the practical context of the two delivered applications and to explain why the final scope differed from the proposal. Because LinkAble ended as two separate but thematically connected applications, analysis had to be performed for both domains without forcing them into an artificial single workflow.

Evidence for the analysis came from the uploaded proposal, the functional and technical specification, the interim report, and the final source archives. The codebase was especially important because it made it possible to distinguish between intended features and completed features.

3.2 Current Context and Existing Situation

Before LinkAble, the practical situation in both problem domains was fragmented. For mobility, a user could rely on conventional route narration or on a human companion, but immediate obstacle description was not guaranteed. For communication, a user could rely on text, video, stickers, or ad hoc explanations, but a stable sign-first workflow remained difficult within ordinary messaging applications. This meant that accessibility often depended on switching between tools, accepting reduced expressiveness, or depending on another person.

The uploaded proposal reflected this fragmentation clearly by originally describing a large integrated accessibility platform. However, the same documents also showed why a narrower implementation was more feasible. The final Vision code supported obstacle awareness rather than comprehensive route intelligence. The final Connect code supported sign-coded communication rather than universal multimodal translation.

3.3 Stakeholders and User Groups

The main stakeholders were the end users of the two applications, the hearing recipients of converted communication, the project supervisors, and the maintainers of sign assets and mappings. A smaller but still relevant stakeholder group was the academic evaluator, because the final report itself had to communicate accurately what had been delivered.

Table 6 - Stakeholder analysis

Stakeholder	Primary interest	Main interaction with the system
Blind and low-vision users	Understand nearby obstacles quickly and non-visually.	Use LinkAble Vision to receive spoken environmental guidance.
SLSL-oriented users	Compose and share sign-based meaning more naturally.	Use LinkAble Connect keyboard, companion screens, and bot conversion.
Hearing chat partners	Receive readable Sinhala output or inspect the sign image.	Read converted bot output in Telegram conversations.

Project supervisor and evaluators	Assess feasibility, delivery quality, and academic honesty.	Review the report and the submitted artefacts.
Maintainers of mappings and sign assets	Keep token-to-language relationships consistent.	Update JSON mappings and sign-image assets when the lexicon changes.

3.4 Feasibility Analysis

A TELOS-style feasibility review was used because the project combined two technically different applications and because the report needed to separate aspirational features from feasible delivery. Technical feasibility was high for the delivered scope because the chosen stacks were lightweight and familiar: React Native with Expo and a FastAPI backend for Vision, and an Android InputMethodService plus a Python Telegram bot for Connect. The main technical risk lay not in creating a user interface but in making the workflows coherent and stable.

Economic feasibility remained moderate to high because the final implementations avoided heavy infrastructure. Vision required a backend and a model runtime but did not depend on a permanently hosted cloud platform for the purposes of academic demonstration. Connect depended on local keyboard logic, PNG assets, and a lightweight bot stack. Operational feasibility was also high because each product followed a clear user journey. Legal and ethical feasibility remained medium because accessibility software interacts with vulnerable user groups and because incorrect guidance or incorrect conversion could create harm.

Table 7 - TELOS feasibility summary

Dimension	Status	Reason
Technical	High	The delivered stacks were realistic and matched the uploaded implementations.
Economic	Medium-High	Limited infrastructure and student-accessible tooling kept cost pressure low.
Operational	High	Both applications followed understandable, bounded workflows.
Legal and ethical	Medium	Accessibility-sensitive output required careful wording, bounded claims, and safe handling of unsupported cases.
Schedule	Medium	Scope reduction improved deliverability, but earlier ambitions created documentation drift.

3.5 SWOT Analysis

A short SWOT analysis clarified the platform position after scope correction.

Table 8 - SWOT analysis

Dimension	Summary
Strengths	Dual accessibility focus, demonstrable software artefacts, deterministic Connect workflow, and clearer academic honesty after report correction.
Weaknesses	Vision remained dependent on backend availability; Connect remained Android-first and mapping-dependent.
Opportunities	Future expansion could add more sign inventory, improved device deployment, and safer route-aware integration for Vision.
Threats	N alerts and evaluation bias could reduce trust if not managed carefully.

3.6 Risk Analysis

Risk analysis was especially important because both applications dealt with accessibility-sensitive communication. For Vision, the main risks were noisy alerts, latency between image capture and response. For Connect, the main risks were token misalignment, incorrect mapping, and ambiguity around how unsupported content was handled.

The final user clarification that token issues and mapping inconsistencies had been corrected was important because it directly addressed one of the core Connect risks. Nevertheless, the report still treated unsupported or unknown content as a continuing risk area. A safe accessibility tool should fail clearly rather than pretend to understand content that lies outside its defined lexicon.

Table 9 - Risk register

Risk	Application	Potential impact	Mitigation
Repeated or over-frequent speech	Vision	Reduced usability and lower trust during movement.	Guidance was phrased briefly and identical messages were suppressed for short intervals.
Backend unavailability	Vision	Loss of detection response during live use.	The app displayed failure messaging instead of silently pretending to detect obstacles.

Token mismatch or mapping conflict	Connect	Incorrect conversions or unreadable communication.	Final hardening aligned token handling and corrected mapping inconsistencies.
Unsupported vocabulary	Connect	Message loss or false meaning.	Known items were converted deterministically and unsupported content was flagged or safely omitted.

Chapter 04 - Requirements Specification

4.1 Overview

The requirements in this chapter were not copied directly from the early proposal or from the intermediate specification. They were corrected to match the delivered scope. This was necessary because some earlier requirements described functions that were no longer evidenced in the final code. The requirements therefore separated Vision requirements, Connect requirements, and shared data or governance requirements.

Requirements were written so that they could be traced either to a code path, an interaction flow, or a final hardening statement supplied by the group. This ensured that the report remained aligned with demonstrable software behaviour.

4.2 Requirements Elicitation and Correction Approach

Three sources were used to formulate the requirements. First, earlier project documents were reviewed in order to understand the original intentions and the mid-stage decisions. Second, the final user clarification about the token issue and mapping consistency was used to reflect the final Connect hardening state. The resulting requirement set therefore represented the intersection between intended purpose and evidenced delivery.

A deliberate distinction was made between complete features and exploratory ideas. For example, the interim documents discussed suggestion models, iOS delivery and route-aware behavior concepts. These topics were useful for understanding project evolution, but they were not all treated as completed final requirements.

4.3 Functional Requirements

4.3.1 LinkAble Vision Functional Requirements

Table 10 - Functional requirements - LinkAble Vision

ID	Requirement statement	Acceptance summary
LV-FR-01	The application shall start with a short countdown and then open the live camera session.	The user reaches a live camera screen after countdown.
LV-FR-02	The application shall capture a frame at controlled intervals and submit it to the backend for analysis.	Frames are submitted without requiring manual capture.
LV-FR-03	The backend shall detect relevant objects and derive direction and approximate distance categories for guidance.	Returned detections contain object label, position, and distance category.

LV-FR-04	The application shall speak backend guidance and suppress immediately repeated messages.	Spoken output is delivered and short-term duplicates are reduced.
LV-FR-05	The application shall expose the latest guidance, the latest detected object, and the current detection state on screen.	Status cards update during live use.
LV-FR-06	The application shall end an inactive session after timeout and allow manual session termination.	A spoken closure message is played and the session ends predictably.

4.3.2 LinkAble Connect Functional Requirements

Table 11 - Functional requirements - LinkAble Connect

ID	Requirement statement	Acceptance summary
LC-FR-01	The system shall provide an Android custom keyboard with alphabetic hand-sign keys and four Sinhala sign pages.	The user can switch pages and select sign keys inside compatible text fields.
LC-FR-02	The keyboard shall insert deterministic sign tokens into the current input connection.	Known keys insert the expected token structure.
LC-FR-03	The keyboard shall provide a visible last-sign preview and control actions for language switching, paging, space, enter, done, and backspace.	Preview and control actions operate during entry.
LC-FR-04	The companion application shall guide keyboard enablement and Telegram access.	The user can open keyboard settings and Telegram from the guide flow.
LC-FR-05	The Telegram bot shall convert known sign tokens into Sinhala text.	Known token input returns readable Sinhala output.
LC-FR-06	The Telegram bot shall convert known Sinhala text into a stitched sign image using sign assets and mapping rules.	Known Sinhala input returns a generated sign image.
LC-FR-07	The final mapping layer shall treat unsupported or ambiguous content safely rather than inventing meaning.	Unknown codes are surfaced clearly or left unrendered with failure feedback.

4.3.3 Shared Platform and Data Requirements

Table 12 - Shared platform and data requirements

ID	Requirement	Verification
SH-FR-01	Mappings and sign-image assets shall be maintained as explicit project artefacts rather than hidden inside the bot logic.	JSON and PNG assets are present in the delivered archive.
SH-FR-02	The token strategy used by the keyboard and bot shall remain deterministic after final hardening.	Token-handling workflow is documented and repeatable.
SH-FR-03	The workflow shall remain user-triggered and shall not claim silent interception of private chats.	Conversion occurs through deliberate keyboard and bot use.

4.4 Non-Functional Requirements

Non-functional requirements were critical because the two applications were meaningful only if they behaved clearly and predictably. In Vision, usefulness depended on timely capture, response, and speech. In Connect, usefulness depended on stable token entry, reliable mapping, and understandable failure behaviour. The non-functional requirements therefore concentrated on responsiveness, clarity, determinism, privacy, and maintainability rather than on inflated enterprise-scale qualities.

Table 13 - Non-functional requirements

Category	Requirement	Applies to
Performance	Interaction delays shall remain low enough for practical demo use and must avoid uncontrolled message queues.	Both
Reliability	Failures such as backend loss or unsupported vocabulary shall produce clear feedback rather than silent behaviour.	Both
Usability	Interfaces shall remain simple, legible, and appropriate for accessibility-focused interaction.	Both
Security and privacy	Only the minimum required external calls shall be made, and message conversion shall remain user-triggered.	Both

Determinism	Known input shall produce the same output when the same mappings and assets are used.	Connect
Maintainability	The code and asset structure shall allow later mapping and feature refinement without rewriting the entire system.	Both
Compatibility	Vision shall run within the delivered Expo workflow and Connect shall operate as an Android IME.	App-specific

4.5 Traceability

The requirements were linked to observable artefacts such as screens, backend services, XML keyboard pages, mappings, and bot responses.

Table 14 - Requirements traceability

Requirement area	Primary evidence type	Example proof
Vision session flow	Screen and behaviour demonstration	Countdown, camera opening, and status cards.
Vision detection logic	Backend code and live response	Object label, direction, and distance category returned through the API.
Connect keyboard workflow	IME source code and device-side demo	Page switching, token insertion, and preview behaviour.
Connect bot workflow	Python bot code and asset library	Token-to-text response and generated sign image.
Mapping hardening	JSON inspection and final user clarification	Token alignment and reverse-mapping consistency.

Chapter 05 - System Architecture

5.1 Introduction

The architecture of LinkAble was not monolithic. The two products shared a project identity and some curated language assets, but they solved different problems and used different technical patterns. Vision followed a mobile-client plus backend-detection model. Connect followed an Android-IME plus Telegram-bot model. The most important architectural correction made in this report was therefore conceptual: the system was described as two coordinated applications rather than as one tightly integrated cloud platform.

This architectural separation improved accuracy. It prevented the report from implying features that the software did not contain and made it easier to explain why each application required its own quality criteria. Vision needed timely capture, backend response, and speech. Connect needed stable token entry, bot parsing, and mapping consistency.

5.2 LinkAble Vision Architecture

LinkAble Vision was delivered as a React Native and Expo mobile frontend paired with a FastAPI backend. The frontend handled countdown, camera activation, periodic capture, UI status display, speech playback, and inactivity timing. The backend handled image decoding, model inference, relevance filtering, prioritisation, and guidance generation. The returned response included a success flag, a primary message, and a list of detections with object label, confidence, direction, distance category, bounding box, and guidance text.

A notable architectural property of Vision was the clear separation between detection and speech. The backend did not speak, it returned text guidance. The frontend then decided whether to vocalise that guidance based on a short duplicate-suppression window.

Another important architectural feature was the controlled capture interval. The frontend captured a frame every 3.5 seconds and blocked new capture when detection was already in progress. For an academic demonstration, this architecture reduced device pressure and simplified state control. The trade-off was that guidance responsiveness depended on the capture interval and the availability of the backend.

Table 15 - Vision architecture components

Component	Purpose	Delivered role
Countdown screen	Provides a short spoken session start and screen transition.	Frontend session entry.
Navigation screen	Hosts the camera, status cards, manual end action, and inactivity timer.	Frontend live interaction layer.
Speech service	Routes spoken English guidance through the device speaker.	Frontend delivery layer.
FastAPI service	Exposes health and detection endpoints.	Backend application layer.

YOLO detection service	Runs object detection and constructs raw detection items.	Backend inference layer.
Guidance service	Converts object position and scale into simple movement advice.	Backend interpretation layer.

5.3 LinkAble Connect Architecture

LinkAble Connect followed a different architecture because its problem was structured communication rather than obstacle awareness. The first layer was the companion Android application. This layer provided the splash screen, setup explanation, keyboard-settings shortcut, Telegram guide, FAQ list, and project information pages. The second layer was the Android custom InputMethodService. This keyboard service loaded XML keyboard definitions, switched between alphabetic and Sinhala panels, inserted tokens into the current input connection, and briefly showed the selected sign in a preview bar. The third layer was the Telegram bot, which converted between sign tokens and Sinhala text or generated a composite sign image from Sinhala input.

The bot architecture was intentionally asset-driven. Two JSON files stored forward and reverse mappings, and a signs folder stored PNG assets. Text conversion from token input used regex parsing and postprocessing. Image generation from Sinhala input used preprocessing to split consonant and vowel-sign patterns, reverse lookup from character to image file, line wrapping, and final image composition through Pillow. This architecture made the system understandable because language logic and image logic were explicit rather than hidden inside a black-box model. The delivered bot responded to user messages, and the delivered image generator returned a single generated image with internal wrapping.

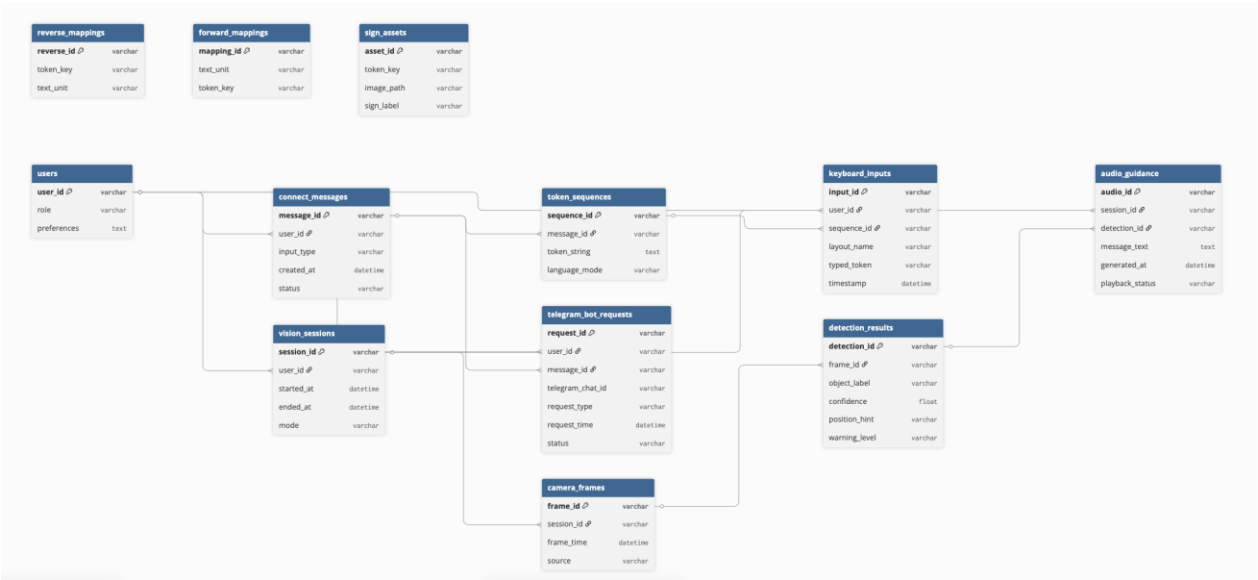


Fig.01 ER Diagram

Table 16 - Connect architecture components

Component	Purpose	Delivered role
Companion launcher app	Guides setup, FAQ use, and Telegram access.	User onboarding and support layer.
HandKeyboardService	Provides custom sign-key entry into any compatible text field.	Android IME layer.
Keyboard XML pages	Define the alphabetic panel and four Sinhala pages.	Static layout and token source.
Preview bar	Shows the most recently selected sign icon.	Immediate selection feedback.
Telegram bot	Receives text messages and returns converted text or sign imagery.	Conversion service layer.
Mapping and asset files	Store the deterministic relationships between codes, characters, and sign images.	Linguistic and visual data layer.

5.4 Technology Stack and Data Assets

The technology selection remained pragmatic and directly visible in the source archives.

Table 17 - Technology stack

Area	Primary technologies	Architectural role
Vision frontend	Expo, React Native, TypeScript, expo-camera, expo-speech, expo-audio.	Mobile camera session, status presentation, and speech output.
Vision backend	FastAPI, Ultralytics YOLO, OpenCV, Pillow, NumPy.	Detection, filtering, and guidance construction.
Connect mobile layer	Android InputMethodService, Kotlin, XML keyboard definitions.	Custom keyboard delivery and setup application.
Connect conversion layer	Python, python-telegram-bot, Pillow, JSON mappings.	Message conversion and sign-image generation.
Shared assets	PNG sign images and explicit mapping files.	Deterministic visual and linguistic reference data.

Table 18 - Data and content assets

Asset	Purpose	Used by
Sign PNG library	Provides the visual vocabulary for keyboard icons and generated sign images.	Connect
Forward mapping file	Maps deterministic sign codes to Sinhala characters or letters.	Connect bot
Reverse mapping file	Maps Sinhala characters back to sign-image filenames.	Connect bot
Keyboard XML layouts	Define sign-key arrangement by page and language panel.	Connect IME
YOLO configuration values	Define model path, confidence threshold, image size, and obstacle limit.	Vision backend
Frontend session logic	Stores countdown, inactivity, and speech-handling behaviour.	Vision app

5.5 Architectural Security, Reliability, and Boundaries

Both applications depended on bounded claims for their safety. Vision was reliable only when the backend was reachable and when camera permission had been granted. Connect was reliable only for the sign inventory defined in the mapping files and only when the user explicitly used the bot for conversion. In both cases, reliability was therefore less about uninterrupted infrastructure and more about avoiding misleading behaviour.

The report treated this as an architectural strength rather than as a weakness. A smaller system that failed honestly was preferable to a larger system that claimed capabilities it did not possess. By separating delivered features from aspirational ones, the architecture chapter also functioned as a quality-control chapter for the report itself.

Chapter 06 - Method of Approach / Methodology

6.1 Introduction

The methodology was structured to guide the development of two complementary products within a single project framework. Its purpose was to support continuous planning, implementation, review, and refinement across both LinkAble Vision and LinkAble Connect. The method emphasized iterative progress, allowing each stage of development to build on validated outputs and documented decisions. This approach ensured that design, implementation, and reporting remained aligned throughout the project lifecycle.

6.2 Development Model Used

An Agile-inspired, milestone-driven development model was followed. The early phase focused on problem definition, scope establishment, requirement identification, and architectural planning. The middle phase concentrated on feature development, technical integration, and refinement of core user workflows for both applications. The later phase focused on system stabilization, testing, evaluation, and final documentation.

This model was appropriate because both applications required repeated review of usability, accessibility, and technical performance during development. The iterative structure supported regular improvement of features such as navigation guidance, obstacle alerts, token-based sign input, preview synchronization, and controlled message conversion. It also enabled steady coordination between implementation work and formal project documentation. As a result, the methodology provided a practical and organized path from initial concept to final deliverable.

Table 19 - Development phases

Phase	Main focus	Outcome
Planning and proposal	Problem framing, background research, and platform ambition.	Initial concept for LinkAble as a dual accessibility project.
Specification refinement	Scope review, architecture planning, and early requirement shaping.	F&TS with a more structured but still ambitious design.
Implementation and interim review	Prototype building, code exploration, and workflow validation.	Working cores for Vision detection and Connect keyboard-plus-bot.
Final correction and packaging	Feature verification, mapping hardening, and report alignment to evidence.	Final report and submission-ready artefacts.

6.3 How the Methodology Was Applied

The methodology was applied through a structured sequence of planning, design, implementation, review, and refinement activities across the project lifecycle. In the early stage, the team established the overall project direction, identified the target user groups, and defined the core objectives of both

LinkAble Vision and LinkAble Connect. During the specification stage, these objectives were translated into clearer functional modules, subsystem responsibilities, and technical workflows.

During implementation, the methodology supported incremental development of the main application features. For LinkAble Vision, this included the integration of image capture, AI-based scene understanding, obstacle awareness, spoken feedback, and location-aware assistance. For LinkAble Connect, it included the development of the structured sign-input keyboard, token mapping logic, preview generation, conversion workflow, and bot-assisted message delivery. Each stage was accompanied by review and refinement so that the applications progressed in a controlled and consistent manner.

In the final stage, the same methodology was applied to verification, documentation alignment, and presentation of the completed system. This ensured that the final report, technical outputs, and implemented features remained consistent with one another and reflected the completed state of the project accurately and professionally.

6.4 Engineering Approach

The engineering approach was lightweight, workflow-oriented, and artefact-led, with strong attention to usability, accessibility, and implementation clarity. Development focused on building complete and reliable interaction flows for both applications so that each system could demonstrate clear end-to-end behaviour. For LinkAble Vision, the engineering work centered on the pipeline from camera input to environmental interpretation and spoken assistance. For LinkAble Connect, the engineering work centered on structured sign entry, deterministic token handling, message previewing, and controlled conversion for communication.

The approach also emphasized explicit technical structures to improve maintainability and traceability. In LinkAble Vision, response handling was organized through structured data fields to support predictable processing and output generation. In LinkAble Connect, sign mappings, token relationships, and media assets were maintained through clearly defined data structures and conversion logic. These engineering choices improved implementation clarity, supported debugging and testing, and contributed to accurate technical documentation in the final report.

6.5 Tools Used in Development and Collaboration

Table 20 - Development and collaboration tools

Tool area	Examples used	Purpose
Mobile development	Expo, React Native, Kotlin, Android Studio-style workflow.	Application implementation and device-side testing.
Backend and bot development	FastAPI, Python, Ultralytics, Pillow, python-telegram-bot.	Detection, conversion, and image generation services.
Asset management	JSON files, PNG libraries, XML keyboard pages.	Maintain mappings, icons, and structured layouts.

Testing and verification	Manual scenario tests, status screens, backend health endpoint, archive inspection.	Functional validation and report correction.
Documentation	Proposal, F&TS, interim report, and final report.	Academic traceability across project stages.

6.6 Team Task Allocation

Task allocation reflected the documentation history supplied in the project artefacts. Problem definition, research gap analysis, requirement analysis, finance discussion, time planning, and external-organisation discussion were distributed across group members in the proposal and interim documents. During software delivery, the two major subsystem streams were Vision implementation and Connect implementation. The final report preserved the documented contribution trail while also recognising that late-stage integration and report correction required cross-team review.

Because the uploaded evidence did not provide a complete per-member code ledger, the team responsibility matrix at the end of the report was presented as a documented responsibility view rather than as an invented quantitative contribution log. This approach protected accuracy while still satisfying the requirement to show fair role distribution.

Chapter 07 - System Testing and Quality Assurance

7.1 Introduction

System testing and quality assurance were important parts of the project because both applications were developed for accessibility-focused use cases where reliability, clarity, and consistency were essential. For LinkAble Vision, testing concentrated on the accuracy and timeliness of environmental feedback, obstacle-related guidance, and spoken output during user interaction. For LinkAble Connect, testing focused on the correctness of token mapping, structured sign input, preview generation, and controlled message conversion.

The testing process was designed to confirm that the implemented workflows operated correctly and that the final systems demonstrated dependable end-to-end behaviour. Quality assurance activities therefore concentrated on functional validation, repeatability of expected outputs, consistency across core features, and alignment between implementation and documentation. This provided confidence that both applications performed according to their intended objectives.

7.2 Testing Strategy

The testing strategy combined code inspection, feature walkthroughs, and scenario-based functional validation. Each major feature was evaluated against its expected behaviour within the complete system workflow. In LinkAble Vision, this included validating the progression from camera initialization and countdown to live scene capture, backend analysis, obstacle-related guidance, and spoken feedback delivery. In LinkAble Connect, this included validating structured keyboard input, deterministic token insertion, preview synchronization, bot-assisted conversion, and message delivery behaviour.

A layered testing model was applied throughout the project. Unit-level testing was used to examine logic related to parsing, token handling, mapping consistency, and response generation. Integration testing was used to verify interaction between connected components, such as frontend capture with backend processing in Vision, and keyboard input with bot conversion services in Connect. System-level testing was then used to evaluate complete user workflows, ensuring that each application functioned as a coherent end-to-end solution.

Table 21 - Testing levels and purpose

Testing level	Purpose	Typical example
Unit reasoning	Check local logic and deterministic rules.	Guidance message construction or token parsing.
Integration testing	Check behaviour across linked components.	Camera capture plus backend response; keyboard plus bot conversion.
System testing	Validate end-to-end user flows.	Starting Vision and hearing output; using Connect inside Telegram.

Failure-path testing	Confirm understandable behaviour under missing support.	Backend unavailable or unsupported vocabulary.
Report verification	Check that documented claims match the artefacts.	Removing unsupported features from the final narrative.

7.3 Vision Testing

Vision testing focused on validating the complete operational workflow of the application. The main objective was to confirm that a user could successfully enter a session, activate the camera, capture image input, receive a structured backend response, and obtain spoken guidance in a clear and timely manner. Additional attention was given to session control behaviours such as inactivity handling and manual exit, ensuring that the overall interaction flow remained stable, predictable, and user-friendly.

Backend testing examined the main service routes and processing logic used to generate guidance. This included validation of the health route, detection route, and decision-making logic that transformed recognition results into concise movement-related assistance. Particular attention was given to detection filtering and prioritization so that the system could present relevant and actionable guidance rather than raw recognition output.

Frontend testing then verified that the processed guidance was presented correctly to the user. This included checking that spoken feedback was triggered appropriately, that repeated messages were controlled within the suppression window to improve usability, and that backend communication failures were communicated clearly through visible interface feedback. Together, these tests confirmed that LinkAble Vision delivered a coherent and reliable end-to-end user experience.

Table 22 - Vision functional test summary

Test area	Objective	Observed or expected outcome
Session startup	Verify countdown and camera activation.	The application moved from countdown to a live camera view without manual navigation.
Frame submission	Verify periodic capture and backend request flow.	Images were submitted at the configured interval when no detection request was already active.
Guidance response	Verify that structured detections became understandable spoken output.	The primary guidance text was displayed and spoken through the device.
Duplicate suppression	Check that immediate repeated guidance was not spoken excessively.	Identical messages were suppressed within the short repeat window.
Inactivity and manual end	Verify controlled session closure.	The application announced closure after inactivity and allowed manual session end.

Backend failure handling	Check clarity when the backend was unreachable.	A backend connection failure message appeared instead of silent behaviour.

7.4 Connect Testing

Connect testing focused on input stability, mapping consistency, and conversion correctness across the main communication workflow. Since the system was designed around deterministic processing, testing emphasized repeatable behaviour and reliable output for defined inputs. The custom keyboard was tested for page navigation, language switching, sign-key selection, backspace, space, enter, and done controls to ensure that user interaction remained stable and predictable during message composition. The preview bar was also tested to confirm that it provided immediate and accurate feedback for the most recently selected sign. In addition, the companion application was tested for successful access to keyboard settings and correct launch behaviour with Telegram integration.

Bot testing covered the two main conversion paths implemented in the system: token-to-Sinhala text conversion and Sinhala-text-to-sign-image generation. For token-based input, testing verified that known token sequences were correctly processed into readable Sinhala text through the defined mapping and post-processing logic. For Sinhala text input, testing verified that the bot generated sign-image outputs in the correct sequence according to the established mappings. These checks confirmed that the conversion workflow operated accurately and consistently across the main supported use cases.

Table 23 - Connect functional test summary

Test area	Objective	Observed or expected outcome
Keyboard enablement	Verify that the user could activate the custom IME.	The companion app opened the relevant settings screen and explained usage.
Token insertion	Verify deterministic keyboard output in text fields.	Known sign keys inserted stable sign tokens.
Preview feedback	Check the visibility of the selected sign.	The most recently selected sign was shown in the preview bar.
Telegram guide flow	Verify that the user could open Telegram and follow the described setup process.	Telegram launched directly or the store page was shown if Telegram was absent.
Token-to-text conversion	Verify bot conversion of known sign codes.	Known codes returned Sinhala text after postprocessing.
Text-to-sign conversion	Verify image generation from known Sinhala input.	Known Sinhala input returned a generated sign image.
Unsupported content handling	Verify safe behaviour for unknown or incomplete mappings.	Unknown items were surfaced, skipped, or failed safely instead of being guessed.

7.5 Quality Assurance Through Report Correction

Quality assurance in this project was applied to both the software products and the final report documentation. In addition to validating implemented system behaviour, the report itself was reviewed carefully to ensure consistency, technical accuracy, and alignment with the completed applications. This review process involved checking feature descriptions, technical explanations, and workflow narratives against the uploaded source code, system outputs, and finalized implementation details. Through this process, the report was refined so that it accurately represented the delivered functionality of LinkAble Vision and LinkAble Connect.

This documentation review was an important part of the overall quality assurance approach because the final report served as the formal academic record of the project. A structured claim-review process was therefore followed to verify that each major system statement was clearly supported by implementation evidence. As a result, the report achieved stronger consistency, improved traceability between description and code, and a more reliable presentation of the final system.

The table below summarises the main report-review and alignment updates carried out during this quality assurance process, showing how the final documentation was refined to match the completed project accurately and professionally.

Table 24 - Evidence-led corrections applied to earlier report claims

Earlier claim from prior documents	Evidence-led correction used in the final report
Turn-by-turn GPS navigation and rerouting were described as delivered Vision features.	The final report described Vision as a camera-based obstacle-awareness application with spoken backend guidance, because GPS route logic was not evidenced in the uploaded final implementation.
An SOS location-sharing flow was previously presented as complete.	SOS was removed from the final delivered feature set because it was not evidenced in the final Vision source archive.
An iOS keyboard extension and Telegram inline operation were discussed as final Connect features.	The final report described an Android IME and a Telegram message bot workflow, because those were the artefacts evidenced by the uploaded code.
A generic cloud core was still implied in the narrative.	The final report removed the claim of a delivered shared cloud platform and instead described a lighter project-level connection based on mappings, assets, and common accessibility intent.

7.6 Limitations of the Final Testing Position

The final testing position was defined around the validated behaviour of the completed system workflows. Testing established confidence in the core operation of both applications by examining functional correctness, repeatability, integration behaviour, and stability across the implemented

features. For LinkAble Vision, this included dependable interaction between the mobile interface, camera capture process, backend communication, structured response handling, and spoken guidance delivery. For LinkAble Connect, this included stable keyboard interaction, deterministic token processing, preview synchronization, and consistent conversion behaviour through the Telegram-based workflow.

The testing position also reflected the technical conditions under which the applications were designed to operate. LinkAble Vision was evaluated as a system that combines mobile capture, AI-supported scene interpretation, and guidance generation through a connected backend workflow. LinkAble Connect was evaluated as a structured communication system built around an explicitly maintained sign inventory, controlled mappings, and predictable conversion logic. This provided a clear and realistic basis for assessing software quality in relation to the actual delivered architecture.

Accordingly, the final report presented the testing evidence through demonstrated workflow coherence, repeatable supported cases, and implementation-aligned quality assurance outcomes. This approach ensured that the testing discussion remained technically accurate, professionally framed, and consistent with the completed products and report expectations.

Chapter 08 - Conclusions

The final report showed that LinkAble was best understood as two bounded accessibility products developed under one project identity. LinkAble Vision delivered a working obstacle-awareness flow that captured images, called a detection backend, and returned spoken movement guidance based on object type, position, and approximate distance. LinkAble Connect delivered a practical sign-input workflow through an Android custom keyboard, a companion setup application, curated sign assets, and a Telegram bot that converted between sign-coded input and user-readable output.

The most important achievement of the final stage was not simply software completion. It was evidential alignment. Earlier stages of the project contained broader plans and stronger claims than the final code could support. By correcting the report against the uploaded source archives and the final user clarification, the project moved from documentation drift to documentation integrity. This made the final submission more academically defensible and more useful to a reader who needed to understand what had actually been built.

The objectives of the project were substantially met within the scope. A camera-driven spoken-awareness application was delivered. A sign-input and bot-mediated communication workflow was delivered. Deterministic mappings and final linguistic hardening were incorporated into the Connect narrative. The report itself was restructured to remove claims that were not evidenced by the final artefacts. The final system therefore succeeded as a realistic academic delivery even though it did not implement every feature envisioned in the proposal.

Several limitations remained. Vision was not a full turn-by-turn navigation system and should not be interpreted as one. Connect was not a universal translation engine and depended on the maintained sign inventory. Neither product should be treated as a complete accessibility replacement for existing support networks or specialist tools. Even so, both applications demonstrated a valuable principle: careful scope reduction, explicit asset management, and honest reporting can produce accessibility software that is smaller than its original ambition but stronger in practical and academic credibility.

Future work could extend Vision with richer on-device performance, broader environmental heuristics, and route-aware integration. Connect could be extended with deeper lexicon expansion, stronger sequence-aware preview, broader device support, and more formally evaluated mapping quality. Those opportunities remained valid precisely because the final delivery established a clearer and more truthful baseline from which later work could grow.

Team Plan and Responsibility Matrix

The submission guide requested a separate table for each member. The tables below reflected the contribution trail documented in the uploaded proposal and interim documents, supplemented only where a general integration or review responsibility was necessary to describe final packaging work.

SAP Prabhashana (10967100)

Name	SAP Prabhashana
Plymouth ID	10967100
Completed tasks	Project description, finance discussion, final report packaging and integration, Vision obstacle-detection and spoken-guidance workflow support, and Connect SLSL mapping plus Android IME core workflow contribution.
Used technologies	React Native/Expo workflow review, FastAPI/YOLO guidance flow understanding, Android IME architecture, Kotlin/XML keyboard structure, JSON sign mappings, and report/document integration tools.
Accrued skills and knowledge	Accessibility-focused system integration, structured requirement-to-implementation alignment, SLSL mapping design, report packaging, and evidence-based technical documentation.
Comments by the group leader	Led the team effectively and ensured the project was completed and submitted in a structured manner.

Rubasinghe D D (10967104)

Name	Rubasinghe D D
Plymouth ID	10967104
Completed tasks	Implementation support, review participation, Vision session/control and dependable-use support, and Connect Android-side application plus backend-related implementation assistance.
Used technologies	Android application workflow, backend integration support, session/state control concepts, mobile interface review, and project testing/review tools.

Accrued skills and knowledge	Application support engineering, dependable interaction-flow design, backend-assisted mobile workflow understanding, collaborative review practice, and implementation validation.
Comments by the group leader	Contributed effectively to development and implementation activities of the project.

YUS De Yapa (10967122)

Name	YUS De Yapa
Plymouth ID	10967122
Completed tasks	Problem statement contribution, Vision audio output and spoken-guidance usability support, and Connect preview-sign workflow support with evaluation/data-analysis contribution.
Used technologies	expo-speech/audio-related workflow concepts, user-feedback and usability evaluation methods, preview-sign interaction logic, and testing/analysis documentation tools.
Accrued skills and knowledge	Accessibility problem framing, voice-UX thinking, usability evaluation, preview-feedback design, and evidence-led feature assessment.
Comments by the group leader	Contributed effectively to development and refinement of the project features.

NPMMD Pathirana (10967063)

Name	NPMMD Pathirana
Plymouth ID	10967063
Completed tasks	External organisation discussion, stakeholder-facing narrative support, Vision UI/UX and accessibility/settings support, and Connect Android app/backend/API support.
Used technologies	React Native/Expo UI concepts, Android application workflow, backend/API integration concepts, accessibility-oriented mobile interface design, and documentation support tools.
Accrued skills and knowledge	Stakeholder-aware communication, UI/UX design for accessibility, settings and interface planning, backend/API coordination, and applied mobile-system analysis.

Comments by the group leader	Contributed well to the reporting workflow analysis and documentation of system requirements.

RRI Dissanayaka (10967081)

Name	RRI Dissanayaka
Plymouth ID	10967081
Completed tasks	Research gap analysis, literature comparison and scope justification, Vision workflow and integration support for the final movement-assistance flow, and Connect preview-sign plus Telegram messaging/conversion workflow support.
Used technologies	Telegram bot workflow, JSON-based conversion logic, preview-sign interaction concepts, integration review methods, and literature-to-system alignment tools.
Accrued skills and knowledge	Research-gap formulation, system justification, workflow integration thinking, mediated messaging-platform design, and sign-first communication analysis.
Comments by the group leader	Contributed effectively to development and refinement of the project features.

AVJA Amarasekera (10967076)

Name	AVJA Amarasekera
Plymouth ID	10967076
Completed tasks	Time frame and planning contribution, methodology and milestone support, Vision permissions/reliability/failure-handling support, and Connect UI/UX support with evaluation/data-analysis contribution.
Used technologies	Project planning tools, mobile permission-handling concepts, reliability and failure-path testing methods, UI/UX evaluation practices, and implementation review workflows.
Accrued skills and knowledge	Project scheduling, failure-aware design, reliability thinking, evaluation planning, and accessibility-oriented interface assessment.

Comments by the group leader	Contributed effectively to development and implementation activities of the project.

PGT Sewmini (10967111)

Name	PGT Sewmini
Plymouth ID	10967111
Completed tasks	Requirement analysis, functional and non-functional requirement structuring, Vision testing/evaluation/release-packaging support, and Connect testing/review support with cross-platform investigation assistance.
Used technologies	Requirements documentation methods, functional test design, scenario-based system testing, release-packaging workflow, and cross-platform review tools.
Accrued skills and knowledge	Requirements engineering, software testing and evaluation, quality assurance documentation, release-readiness review, and structured verification practice.
Comments by the group leader	Contributed effectively to development and refinement of the project features.

Reference List

American Speech-Language-Hearing Association (ASHA) (2023) *Augmentative and Alternative Communication (AAC).* Available at: https://www.asha.org/Practice-Portal/Professional-Issues/Augmentative-and-Alternative-Communication/ (Accessed: 15 October 2025). (ASHA)

Elly, B. and Ahsun, A. (2024) *Deep Learning for Real-time Object Detection and Tracking in Surveillance Systems.* Available at: https://www.researchgate.net/publication/385885536_Deep_Learning_for_Real-time_Object_Detection_and_Tracking_in_Surveillance_Systems (Accessed: 17 October 2025)

Microsoft Research (2022) *Soundscape: Audio Navigation for the Blind.* Available at: https://blogs.microsoft.com/accessibility/soundscape/ (Accessed: 20 October 2025). (The Official Microsoft Blog)

Olusanya, B.O., Davis, A.C. and Hoffman, H.J. (2019) 'Hearing loss: rising prevalence and impact', *Bulletin of the World Health Organization*, 97(10), pp. 646–646A. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC6796666/ (Accessed: 31 October 2025).

Tager-Flusberg, H. and Kasari, C. (2022) 'Nonverbal individuals with autism spectrum disorder: research review and future directions', *Autism Research*, 15(2), pp. 191–206. **Available at**: https://onlinelibrary.wiley.com/doi/10.1002/aur.1329. (Accessed: 10 November 2025).

World Health Organization (WHO) (2019) *World report on vision.* Geneva: World Health Organization. Available at: https://www.who.int/Publications/i/Item/9789241516570 (Accessed: 24 November 2025). (World Health Organization)

World Health Organization (WHO) (2024) *Deafness and hearing loss.* Available at: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss (Accessed: 27 November 2025). (World Health Organization)

Kurupita, A. (2021) *Sri Lankan Sign Language.* Available at: https://www.researchgate.net/publication/356375091_Sri_Lankan_Sign_Language . (Accessed: 12 December 2025)

Punchimudiyanse, M. and Meegama, R.G.N. (2017) 'Animation of fingerspelled words and number signs of the Sinhala Sign Language', *ACM Transactions on Asian and Low-Resource Language Information Processing*, 16(4), Article 24, pp. 1–26. Available at: https://doi.org/10.1145/3092743 (Accessed: 22 December 2025).

Kumarawadu, P. and Izzath, M. (2022) 'Sinhala Sign Language Recognition using Leap Motion and Deep Learning', *Journal of Artificial Intelligence and Capsule Networks*, 4(1), pp. 54–68. doi: 10.36548/jaicn.2022.1.004. Available at: https://www.researchgate.net/publication/360000380_Sinhala_Sign_Language_Recognition_using_Leap_Motion_and_Deep_Learning(Accessed: 4 January 2025)

Appendices

Appendix A - Source Code Submission

Group 80 - LinkAble Vision Source Code

Group 80 - LinkAble Connect Source Code

Appendix B – Github Source Code

https://github.com/Pawan-Prabhashana/Group-80-LinkAble-Connect

https://github.com/Pawan-Prabhashana/Group-80-LinkAble-Vision

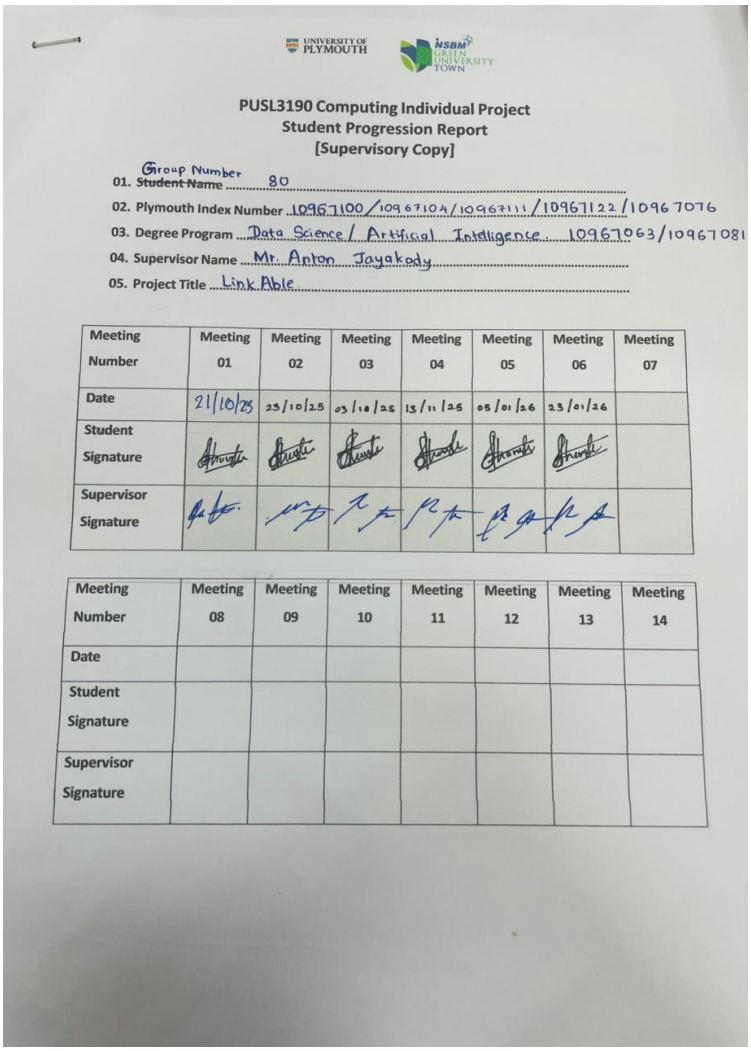
Appendix C – Previous Submissions

80_Proposal.pdf

Group_80_Functional
Technical Specificatior

Group_80_Interim.pdf

Appendix D – Signing sheet



UNIVERSITY OF PLYMOUTH

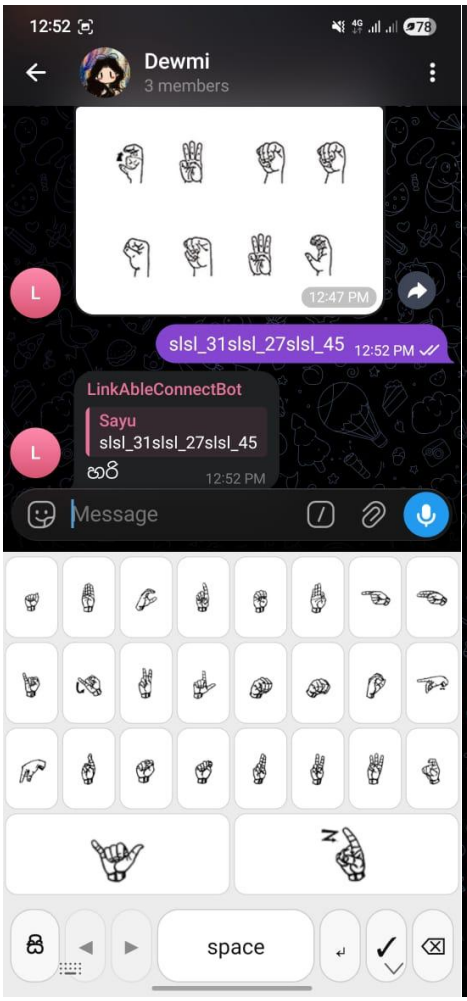
NSBM GREEN UNIVERSITY TOWN

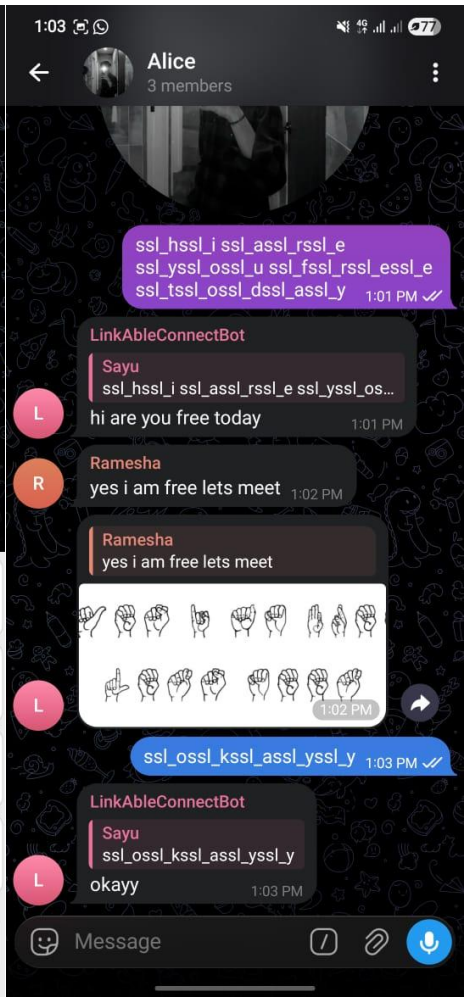
PUSL3190 Computing Individual Project
Student Progression Report
[Supervisory Copy]

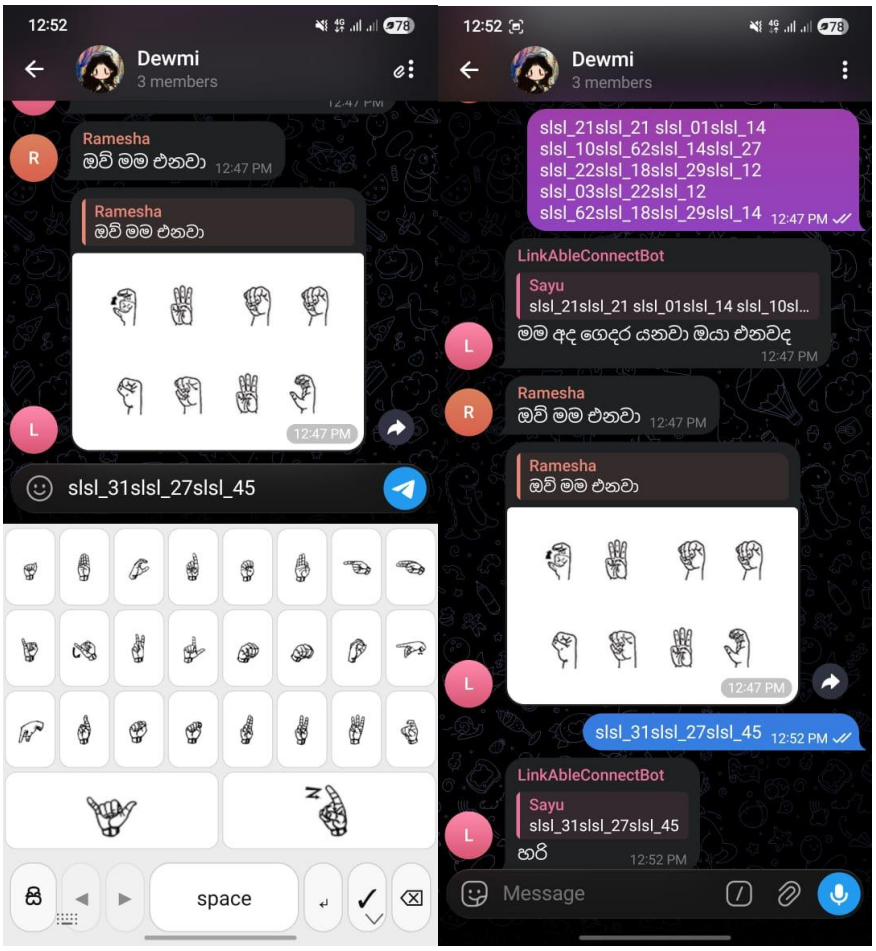
01. ~~Student Name~~ Group Number 80
02. Plymouth Index Number ..10967100/10967104/10967111/10967122/10967076
03. Degree Program ...Data Science / Artificial Intelligence... 10967063/10967081
04. Supervisor Name ...Mr. Anton Jayakody.....
05. Project Title ...Link Able.....

Meeting Number	Meeting 01	Meeting 02	Meeting 03	Meeting 04	Meeting 05	Meeting 06	Meeting 07
Date	21/10/25	23/10/25	03/10/25	13/11/25	05/01/26	23/01/26	
Student Signature							
Supervisor Signature							

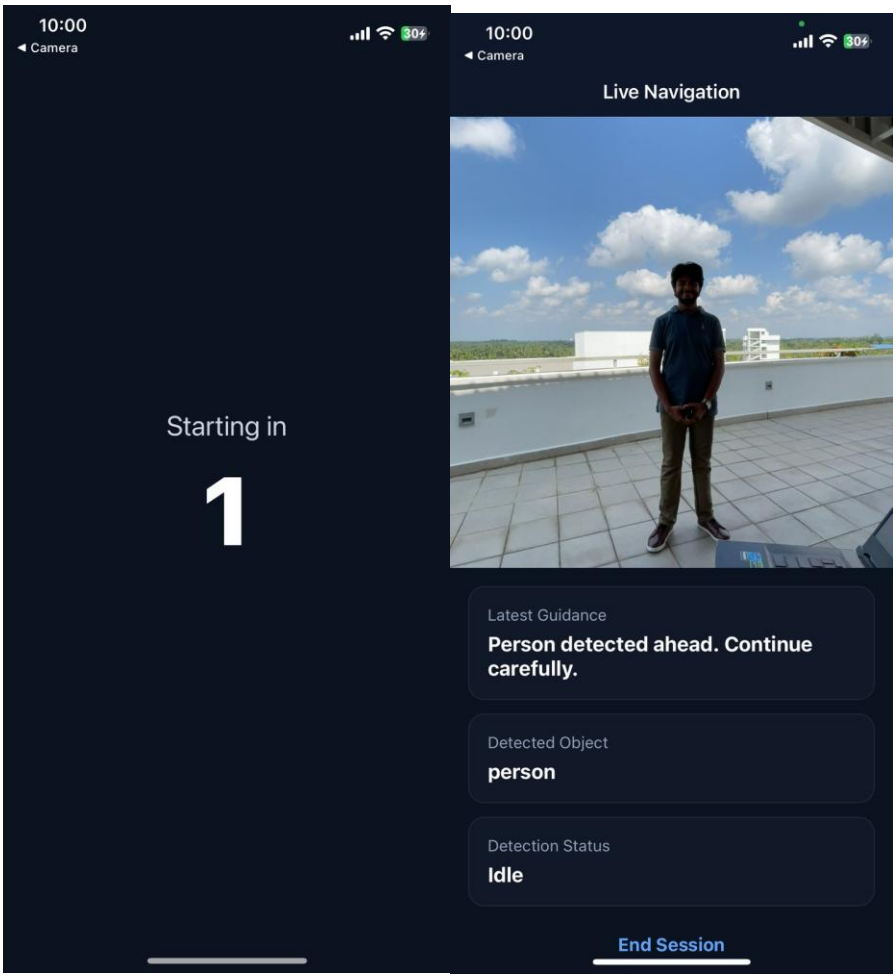
Meeting Number	Meeting 08	Meeting 09	Meeting 10	Meeting 11	Meeting 12	Meeting 13	Meeting 14
Date							
Student Signature							
Supervisor Signature							

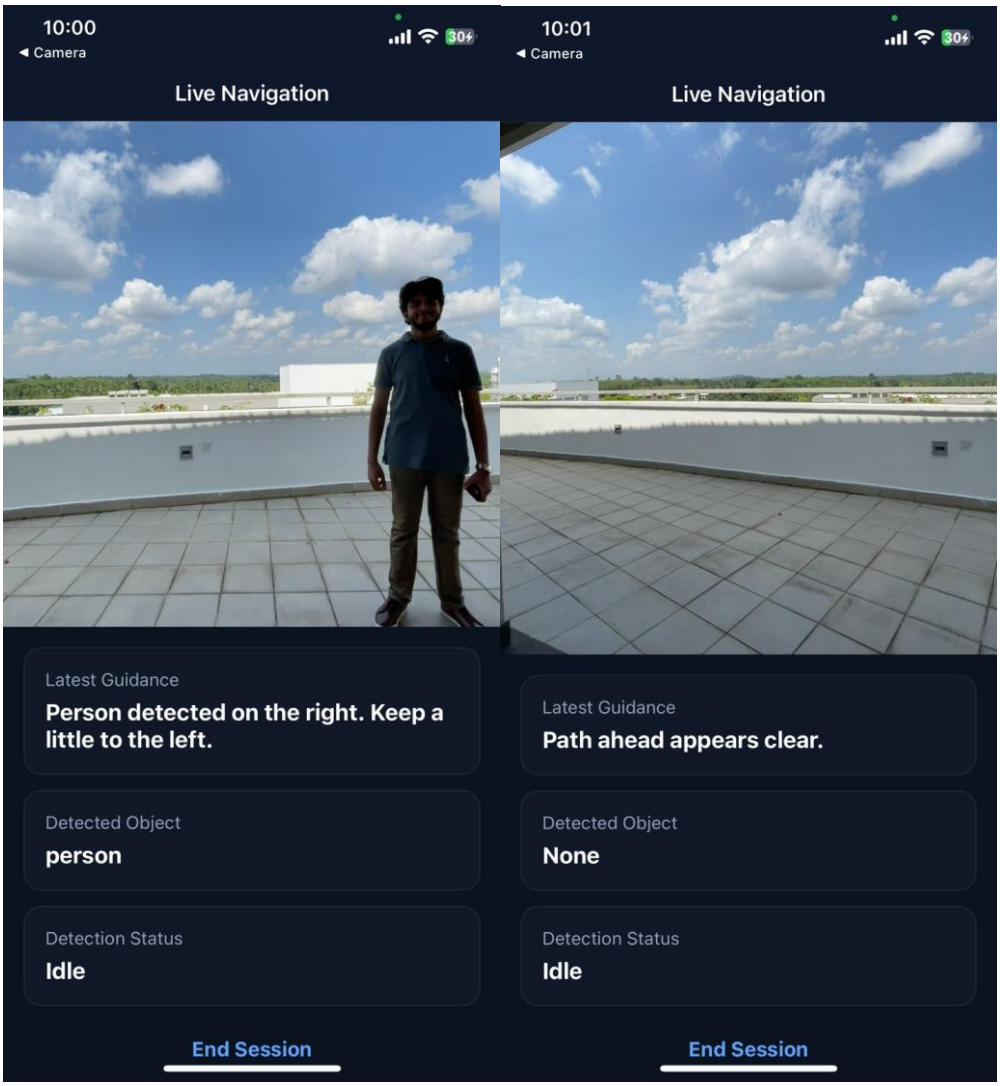

12:52
Dewmi
3 members
12:47 PM
slsl_31slsl_27slsl_45 12:52 PM
LinkAbleConnectBot
Sayu
slsl_31slsl_27slsl_45
හරි
12:52 PM
Message
space

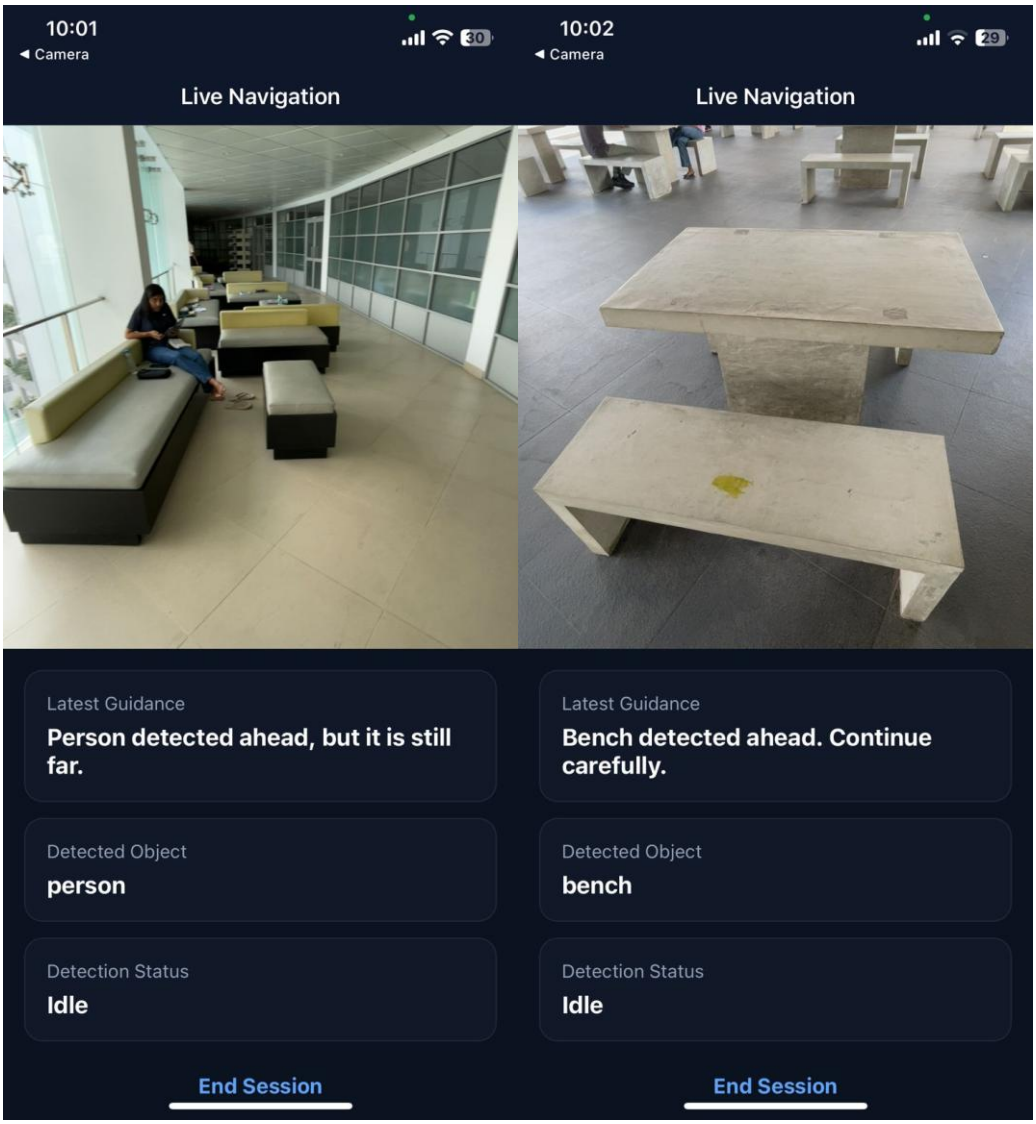

1:03
Alice
3 members
ssl_hssl_i ssl_assl_rssl_e
ssl_yssl_ossl_u ssl_fssl_rssl_essl_e
ssl_tssl_ossl_dssl_assl_y 1:01 PM
LinkAbleConnectBot
Sayu
ssl_hssl_i ssl_assl_rssl_e ssl_yssl_os...
hi are you free today
1:01 PM
Ramesha
yes i am free lets meet 1:02 PM
Ramesha
yes i am free lets meet
1:02 PM
ssl_ossl_kssl_assl_yssl_y 1:03 PM
LinkAbleConnectBot
Sayu
ssl_ossl_kssl_assl_yssl_y
okayy
1:03 PM
Message


12:52
Dewmi
3 members
Ramesha
ඔව් මම එනවා 12:47 PM
Ramesha
ඔව් මම එනවා
12:47 PM
slsl_31slsl_27slsl_45
space
12:52
Dewmi
3 members
slsl_21slsl_21 slsl_01slsl_14
slsl_10slsl_62slsl_14slsl_27
slsl_22slsl_18slsl_29slsl_12
slsl_03slsl_22slsl_12
slsl_62slsl_18slsl_29slsl_14 12:47 PM
LinkAbleConnectBot
Sayu
slsl_21slsl_21 slsl_01slsl_14 slsl_10sl...
මම අද ගෙදර යනවා ඔයා එනවද
12:47 PM
Ramesha
ඔව් මම එනවා 12:47 PM
Ramesha
ඔව් මම එනවා
12:47 PM
slsl_31slsl_27slsl_45 12:52 PM
LinkAbleConnectBot
Sayu
slsl_31slsl_27slsl_45
හරි 12:52 PM
Message

Appendix E - Snapshots of the Vision App




10:00
Camera
Live Navigation
Latest Guidance
Person detected on the right. Keep a little to the left.
Detected Object
person
Detection Status
Idle
End Session
10:01
Camera
Live Navigation
Latest Guidance
Path ahead appears clear.
Detected Object
None
Detection Status
Idle
End Session


10:01
Camera
Live Navigation
Latest Guidance
Person detected ahead, but it is still far.
Detected Object
person
Detection Status
Idle
End Session
10:02
Camera
Live Navigation
Latest Guidance
Bench detected ahead. Continue carefully.
Detected Object
bench
Detection Status
Idle
End Session